



Objetivos

- Practicar las técnicas básicas de definición de funciones en un lenguaje funcional.
- Diseñar funciones de forma recursiva para el tratamiento de listas.
- Conocer las funciones de orden superior básicas para tratamiento de listas.

Tarea

Diseña e implementa en Haskell las funciones que se piden a lo largo de la práctica.

Importante

Implementa las funciones que se piden en la Tarea 1 ('tplot') y en la Tarea 2 ('lsystem'), junto con todo el código que necesite su implementación, en un **módulo independiente y autocontenido**, llamado 'LSystem', (y por tanto implementado en un fichero fuente llamado 'LSystem.hs'), de forma que pueda ser usado desde otro programa principal distinto al tuyo.

1. Gráficos de Tortuga

El sistema denominado de **Gráficos de Tortuga** es un entorno programable de dibujo vectorial, popularizado por el lenguaje de programación **Logo**, que permite generar gráficos 2D basados en líneas poligonales mediante el desplazamiento de un cursor: la *tortuga* de Logo.

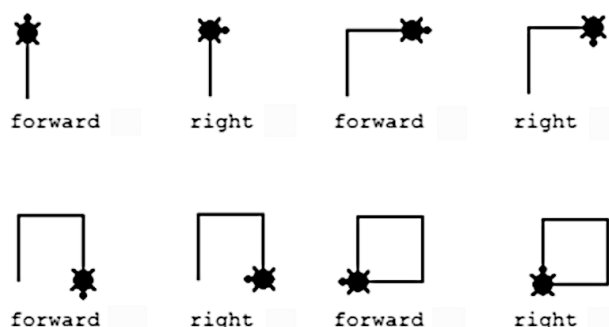
La **tortuga** se define mediante una posición y una orientación en el plano:

- la **posición** se define mediante un punto con dos coordenadas cartesianas (x, y) .
- la **orientación** se define mediante un ángulo en grados medido desde el eje X en sentido anti-horario.

La tortuga puede moverse para trazar líneas, controlada mediante órdenes sencillas:

- **avanzar** hacia adelante, en la dirección que indica la orientación de la tortuga, una distancia predeterminada (*paso*), trazando una línea.
- **girar** la orientación de la tortuga hacia la derecha o la izquierda un valor predeterminado (*giro*).

Por ejemplo, para dibujar un cuadrado, una vez fijados el paso (1 unidad), el ángulo de giro (90° en este caso), la posición inicial de la tortuga (el punto $(0, 0)$) y su orientación inicial (90° , vertical hacia arriba), el proceso sería el siguiente:



(el último giro no sería estrictamente necesario).

En el entorno de programación del lenguaje Logo el dibujo se hace directamente sobre una ventana, escribiendo un programa que controla la tortuga mediante órdenes del lenguaje. En esta práctica, para simplificar, generaremos una línea poligonal que se puede escribir en un fichero con formato SVG, que podrás visualizar abriéndolo en el mismo VS Code o en tu navegador.

El programa que controlará la tortuga se definirá mediante órdenes que serán simplemente caracteres en una cadena de texto. Partiendo de un estado inicial de la tortuga, y con un paso y un giro fijados, cada carácter se interpretará como un comando para la tortuga, que generará un nuevo estado (nueva posición u orientación) de la tortuga:

- '>': avanza la tortuga hacia delante
- '+': gira la tortuga hacia la derecha (sentido horario)
- '-': gira la tortuga hacia la izquierda (sentido anti-horario)

En el módulo `Turtle.hs` puedes ver las definiciones de los tipos de datos y funciones que te damos para manejar la tortuga. Una tortuga (tipo de datos `Turtle`) se define mediante una tupla que contiene la información del paso de avance y del ángulo de giro (en grados) predefinidos, y la posición y orientación actuales.

Para generar el gráfico, se debe generar una secuencia de datos de tipo `Position`, y escribirla en un fichero SVG mediante la función `'saveSvg "nombre" puntos'`, definida en el módulo `SVG.hs`.

De esa forma, para dibujar el cuadrado del ejemplo anterior, la información a utilizar sería la siguiente:

- Definición inicial de la tortuga: `(1, 90, (0,0), 90)`
- Secuencia de comandos: `">+>+>+>+"`

1.1. Tarea 1

Implementa en el módulo `LSystem` una función Haskell llamada `'tplot'` que, a partir del estado inicial de la tortuga y una secuencia de comandos almacenados en una cadena, devuelva la lista de puntos que corresponde al gráfico generado:

```
tplot :: Turtle -> String -> [Position]
```

Utilízala desde tu programa principal para generar distintas figuras geométricas (triángulo, cuadrado, círculo) y grábalas en formato SVG mediante la función `'saveSvg'` disponible en el módulo `SVG`. Por ejemplo, para el cuadrado anterior:

```
figura = tplot (1,90,(0,0),90) ">+>+>+>+"
-- figura: [(0.0,0.0),(0.0,1.0),(1.0,1.0),(1.0,0.0),(0.0,0.0)]
main = do
    saveSvg "cuadrado" figura
```

Nota importante: No debes modificar los módulos `Turtle` o `SVG` (y por tanto, los ficheros `Turtle.hs` o `SVG.hs`), **bajo ninguna circunstancia**. De hecho, pueden ser reemplazados por los originales en el momento de la evaluación de la práctica.

Recuerda que debes probar el código implementado de forma exhaustiva.

2. Sistemas de Lindenmayer

Los **Sistemas de Lindenmayer** o **L-Systems** son gramáticas formales que, mediante ciertas reglas de re-escritura, se utilizan para modelar de forma recursiva la geometría de elementos naturales, como plantas, una línea de costa o un copo de nieve, y representarlas mediante un sistema de gráficos de tortuga.

Un L-System añade al alfabeto de símbolos de la tortuga ('>', '+', '-') otros símbolos nuevos, que pueden ser letras mayúsculas o minúsculas. El sistema parte de un cadena inicial (*axioma*), que puede re-escribirse reemplazando cada una de las letras por una nueva cadena de símbolos. La cadena que sustituye a cada símbolo se define mediante una serie de *reglas de re-escritura*. Ese proceso se repite un número determinado de veces, generando una cadena final, que se interpretará para generar la geometría.

Una regla de re-escritura especifica cómo se sustituye cada símbolo, por ejemplo:

$$F \rightarrow F+F+F$$

Los símbolos para los que no se defina una regla se conservan sin cambios. Normalmente los símbolos originales de control de la tortuga no se reescriben.

Para generar el gráfico, la interpretación final de los símbolos es la siguiente:

- '>', '+', '-': mueven la tortuga como se ha descrito anteriormente.
- letra mayúscula ['A' . . 'Z']: avanza la tortuga, de manera similar a '>'.
- letra minúscula ['a' . . 'z']: se ignora a la hora de dibujar, sólo se utiliza en el proceso de re-escritura.

A partir de un axioma inicial y una serie de reglas, podemos obtener la definición de nuestro gráfico repitiendo la aplicación de las reglas. En cada sucesiva aplicación se sustituye cada símbolo de la cadena según la regla correspondiente (en el caso de que exista, en caso contrario se conserva sin cambios).

2.1. Tarea 2

Utilizando el lenguaje Haskell, implementaremos la definición de las reglas de re-escritura mediante una función con un interfaz similar al siguiente:

```
rules :: Char -> String
```

que devolverá la cadena que reemplaza a un símbolo dado. Cada figura geométrica se definirá mediante una función con distinto nombre, pero manteniendo el mismo interfaz. Las definiciones de las reglas para cada figura estarán en tu programa principal.

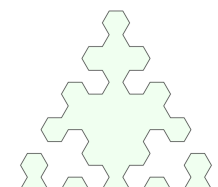
Implementa en el módulo LSystem la función 'lssystem' que, recibiendo como entrada (parámetros) una función con el interfaz anterior (que define las reglas de re-escritura), el axioma inicial y el número de veces que tenemos que aplicarlas, devuelva la cadena resultante:

```
lssystem :: (Char -> String) -> String -> Int -> String
```

Si para la implementación de esta función deseas definir otras funciones auxiliares, estás deberán estar también implementadas en el módulo LSystem, de forma que éste sea autocontenido.

Por ejemplo, para la figura conocida como *Sierpinsky Arrowhead*, cuya definición es la siguiente:

Reglas: $F \rightarrow G+F+G$
 $G \rightarrow F-G-F$
 Axioma: F
 Paso: 1
 Angulo: 60 grados



(la figura que se muestra es la obtenida al aplicar 4 veces las reglas) podemos implementar la definición de sus reglas de re-escritura mediante una función

```
rulesArrowhead :: Char -> String
-- 'F' -> "G+F+G"
-- 'G' -> "F-G-F"
-- otros ?
```

que nos permite obtener versiones cada vez más detalladas de nuestra figura, aplicándola un número creciente de veces:

```
> lsystem rulesArrowhead "F" 0
"F"
> lsystem rulesArrowhead "F" 1
"G+F+G"
> lsystem rulesArrowhead "F" 2
"F-G-F+G+F+G+F-G-F"
> lsystem rulesArrowhead "F" 3
"G+F+G-F-G-F-G+F+G+F-G-F+G+F+G-F-G-F+G+F+G"
```

Implementa en tu programa principal diversas instancias de la función 'rules', con distintos nombres, que definan distintos L-Systems. Genera los ficheros SVG correspondientes a cada uno de ellos. En el Apéndice tienes la definición de algunos sistemas conocidos, y puedes encontrar muchos otros en Internet.

Recuerda que debes probar el código implementado de forma exhaustiva, y que las funciones que se te piden deben cumplir exactamente el interfaz propuesto. En este caso **el programa principal que generes te ayudará a verificar que tu código funciona correctamente**, pero el módulo LSystem puede ser probado desde otro programa principal que siga las especificaciones de la práctica.

Entrega

Como resultado de esta práctica deberás entregar todos los ficheros de código fuente que hayas necesitado para resolver el problema, incluyendo tu programa principal `main.hs` y los ficheros suministrados como material de partida.

Recuerda que en esta práctica:

- Todo tu código se debe cargar desde un **único módulo llamado LSystem**.
- Las funciones `tpPlot` y `lsystem` deben tener los **nombres** y cumplir los **interfaces especificados**.
- **No debes modificar** los módulos `Turtle` y `SVG`.

La solución deberá ser compilable y ejecutable con los ficheros que has entregado mediante los siguientes comandos:

```
1 ghc --make main.hs
2 ./main
```

No se deben utilizar paquetes, bibliotecas, ni ninguna infraestructura adicional fuera de las bibliotecas estándar del propio lenguaje.

En caso de no compilar siguiendo estas instrucciones, **la nota de la evaluación de la práctica será un 0**.

Todos los archivos de código fuente (y directorios, si es el caso) solicitados en este guión deberán ser comprimidos en un único archivo ZIP con el siguiente nombre:

- **practica5_<nip1>_<nip2>.zip** (donde <nip1> y <nip2> son los NIPs de los estudiantes involucrados) si el trabajo ha sido realizado en pareja.
En este caso sólo uno de los dos estudiantes deberá hacer la entrega en Moodle.
- **practica5_<nip>.zip** (donde <nip> es el NIP del estudiante involucrado) si el trabajo ha sido realizado de forma individual.

El archivo comprimido a entregar no debe contener ningún fichero aparte de los fuentes (y en su caso el fichero `Makefile`) que te pedimos: ningún fichero ejecutable (*.exe) u objeto (*.o), clases compiladas de Java (*.class), ficheros de interfaz de Haskell (*.hi), archivos de configuración del entorno de desarrollo (.vscode), ni ningún otro fichero adicional.

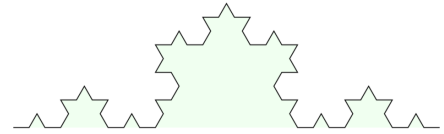
La entrega del archivo ZIP deberá hacerse en la tarea correspondiente preparada en el curso Moodle de la asignatura y antes de la fecha límite fijada para ello.

Apéndice: Definición de diversos L-Systems

En todos los ejemplos se usa un paso de la tortuga igual a 1.

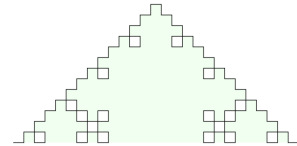
Curva de Koch

Reglas: $F \rightarrow F-F++F-F$
 Axioma: F
 Angulo: 60 grados
 Iteraciones: 3



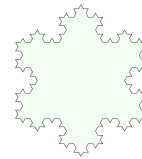
Curva de Koch cuadrada

Reglas: $F \rightarrow F-F+F-F$
 Axioma: F
 Angulo: 90 grados
 Iteraciones: 3



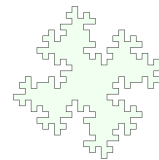
Koch Snowflake

Reglas: $F \rightarrow F-F++F-F$
 Axioma: $F++F++F$
 Angulo: 60 grados
 Iteraciones: 3



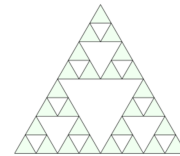
Isla de Minkowski

Reglas: $F \rightarrow F-F+F+FF-F-F+F$
 Axioma: $F+F+F+F$
 Angulo: 90 grados
 Iteraciones: 2



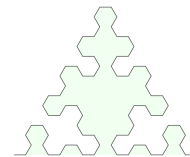
Triángulo de Sierpinsky

Reglas: $F \rightarrow F-G+F+G-F$
 $G \rightarrow GG$
 Axioma: $F-G-G$
 Angulo: 120 grados
 Iteraciones: 3



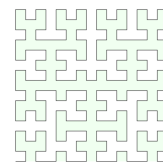
Sierpinsky Arrowhead

Reglas: $F \rightarrow G+F+G$
 $G \rightarrow F-G-F$
 Axioma: F
 Angulo: 60 grados
 Iteraciones: 4



Curva de Hilbert

Reglas: $f \rightarrow -g>+f>+>g-$
 $g \rightarrow +f>-g>g->f+$
 Axioma: f
 Angulo: 90 grados
 Iteraciones: 4



Curva de Gosper

Reglas: $F \rightarrow F-G--G+F++FF+G-$
 $G \rightarrow +F-GG--G-F++F+G$
 Axioma: F
 Angulo: 60 grados
 Iteraciones: 3

